EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 2 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE I · DATOS CON GARANTÍAS

Datos con garantías

Preguntar cualquier cosa a los datos y fiarse de la respuesta; medirlo y elegir.

Pregunta guía
El SQL libre respondía a todo y fallaba sin avisar: ¿cómo lo hacemos bien?

EN ESTE BLOQUE

1 **El SQL libre, bien hecho**
Capa semántica, un modelo capaz y un verificador.

2 **Cómo se mide**
Un examen con respuestas calculadas a mano.

3 **La nota y el coste**
De 58 % a 91 %; memoria, velocidad y el modelo de expertos.

4 **Comparar y cerrar**
Qué elegimos y cuatro frases para dirección.

Dónde lo dejamos: tres formas de dar los datos al modelo

`04_conectar_ollama.py`

El modelo redacta

QUIÉN CALCULA

nuestro código (el 03), antes de preguntar

QUÉ SE PUEDE PREGUNTAR

solo lo que calculamos antes

✓ Fiable ✗ Limitado

`07_herramienta_funcion.py`

Función fija

QUIÉN CALCULA

nuestro código (la función)

QUÉ SE PUEDE PREGUNTAR

lo que cabe en la función

✓ Fiable ✗ Limitado

`08_herramienta_sql.py`

SQL libre

QUIÉN CALCULA

el modelo, escribiendo el SQL

QUÉ SE PUEDE PREGUNTAR

cualquier cosa

✓ Libre ✗ Falla sin dar error

Hoy: **libertad para preguntar** cualquier cosa y **fiabilidad** en la respuesta, a la vez.

EBIS Business Techschool

PRÁCTICA 1 · BLOQUE I

El SQL libre, bien hecho

La misma pregunta que falló en la sesión 1: primero como entonces y después con la capa semántica.

▸ Ahora, en la terminal

AL TERMINAR

Las slides siguientes explican por qué fallaba y qué probamos.

PASOS

1 **La pregunta de la sesión 1, como entonces**

```
python practica/sesion_2/01_preguntar_datos.py --modo clase
```

→ Tablas crudas y reglas, con un modelo de 30B: 39.817,63 €. Sigue fallando.

2 **Lo que verá ahora el modelo**

```
python practica/sesion_2/02_capa_semantica.py
```

→ Vistas con comentarios. Los seis CSV siguen ahí, pero no se incluyen en el esquema que presentamos al modelo.

3 **La misma pregunta, con la capa**

```
python practica/sesion_2/01_preguntar_datos.py
```

→ Una sola consulta: 40.000 y 26.000 €. Y el código revisa cada cifra.

4 **Una pregunta que nadie había previsto**

```
python practica/sesion_2/01_preguntar_datos.py -q "¿Qué ventas se
hicieron en las tres tiendas el 25 de abril?"
```

→ Separado por tienda, en una consulta.

Por qué fallaba: el negocio está escondido en los datos crudos

1

¿Dónde acaba el periodo A?

Tomó todo marzo

42.332,73 €

real: 40.000

2

Unir tablas multiplica filas

Unió ventas con el calendario: cada venta, 28 veces

1.962.800 €

real: 40.000

3

Un abono cuenta en su fecha

Restó los abonos por la fecha de la venta

39.817,63 €

real: 40.000

4

Contar el pedido entero

Miró cada línea (cantidad ≥ 4), no el pedido

0 pedidos

real: 108

5

Un pedido no es un cliente

Contó pedidos como personas

120 clientes

real: no existe

Nada de esto se ve mirando las columnas. Con reglas en el prompt, el modelo puede leerlas… **e ignorarlas**.

Probamos seis palancas, una a una, con un examen

Palanca	Qué cambia	Resultado (13 preguntas)
Un modelo más grande	Del 4B al 30B, con el mismo prompt	8 → 8 ✗ no mejora
El diccionario de la empresa	El documento oficial, en el prompt	8 → 9 ≈ poco
Ejemplos resueltos	Tres preguntas con su SQL	12 frente a 13 ≈ no ayuda
Razonar antes de responder	gpt-oss:20b, con tablas crudas	12, pero 78 s por pregunta ≈
Capa semántica	**Vistas con el negocio dentro**	**8 → 13 ✓ la palanca grande**
Verificador en el código	Revisa las cifras de la respuesta	Caza cifras inventadas ✓

Examen de 13 preguntas con respuestas calculadas a mano (cómo se hace: en «Comparar y cerrar»). Modelo de 30B.

La capa semántica: el negocio, escrito una vez en código

CSV · ESQUEMA `crudo`

- ventas.csv
- reembolsos.csv
- referencias.csv
- locales.csv
- calendario.csv
- disponibilidad.csv

→

VISTAS · ESQUEMA PRESENTADO AL MODELO

resumen_diario	por tienda y día: pedidos, unidades, ventas, abonos y ventas netas
pedidos	una fila por compra: líneas, unidades e importe
ventas_lineas	cada línea con su periodo, tienda y producto ya pegados
abonos	periodo por fecha del abono; tienda heredada de la venta
movimientos	ventas (+) y abonos (−) juntos, por producto
disponibilidad	stock con producto y horas; solo 8 SKU

→

El modelo

Escribe SQL sobre las vistas.

Casi nunca necesita un JOIN.

El modelo ya no tiene que **saber el negocio**: solo **traducir la pregunta**.

PRÁCTICA 1 · 02_capa_semantica.py

Lo que falló en la sesión 1, ahora en una consulta

SESIÓN 1 · TABLAS CRUDAS + REGLAS

```
SELECT SUM(v.importe_linea) FROM ventas v
WHERE v.fecha_venta >= '2026-03-02'
  AND v.fecha_venta <  '2026-04-01'    -- todo marzo
  AND v.local_id = 'MAD-MAL'

SELECT SUM(r.importe_reembolso) FROM reembolsos r
JOIN ventas v ON r.linea_id = v.linea_id WHERE …
-- …y del periodo B, nada
```

Dos consultas · **42.332,73 €** y sin periodo B ✗

HOY · CAPA SEMÁNTICA

```
SELECT periodo, SUM(ventas_netas)
FROM resumen_diario
WHERE tienda = 'Malasaña'
GROUP BY periodo

-- el periodo, los abonos por su fecha y
-- la tienda de cada abono, ya están en la vista
```

Una consulta · **40.000 €** y **26.000 €** ✓

Sin JOIN y sin fechas que equivocar: lo difícil se escribió una vez, en las vistas, y se comprobó contra el 03.

PRÁCTICA 1 · 01_preguntar_datos.py

Cómo lo hacemos bien: tres piezas

1

Capa semántica

El negocio, escrito una vez en código: al modelo se le presenta el esquema de vistas.

`02_capa_semantica.py`

2

Un modelo capaz

qwen3:30b-a3b: entiende la pregunta y, por ser de expertos, va rápido.

`comparar y cerrar: nota y coste`

3

Un verificador

El código revisa que cada cifra de la respuesta salga de un resultado de SQL.

`01_preguntar_datos.py`

Y lo de siempre: el modelo **escribe** el SQL; nuestro código lo **ejecuta** con barreras… y el SQL **se enseña**.

Dónde vive el conocimiento del negocio

Forma	Dónde está el negocio	Qué se puede preguntar	Fiabilidad
04 · El modelo redacta	En el código (el 03)	Solo lo previsto	Máxima
07 · Función fija	En el código (la función)	Lo que cabe en la función	Alta
08 · SQL libre	En el prompt (reglas)	Cualquier cosa	Baja (~60 %)
Capa semántica	**En el código (las vistas)**	**Cualquier cosa**	**Alta (~90 %)**

El negocio vuelve **al código**… sin quitarle al modelo **la libertad de preguntar**.

EBIS Business Techschool

PRÁCTICA 2 · BLOQUE I

Medir: nota y coste

Ponerle nota a lo de la sesión 1 y a lo de hoy, y ver cuánto cuesta cada modelo.

▶ Ahora, en la terminal

AL TERMINAR
Las slides siguientes explican el examen, la nota y el coste.

PASOS

1 El monitor, en otra terminal

```
python practica/sesion_2/03_monitor_recursos.py
```

→ RAM, GPU y CPU del Jetson en vivo. Antes: ollama ps (y ollama stop si hay algo).

2 El modelo de la sesión 1

```
python practica/sesion_1/04_conectar_ollama.py
```

→ El 4B: la RAM sube unos 3 GB y escribe a unos 7 tokens por segundo.

3 Examen: el SQL de la sesión 1

```
python practica/sesion_2/04_examen.py --modo clase --rapido
```

→ Carga el 30B (la RAM sube 18 GB). Cinco preguntas con respuesta a mano: 2 de 5.

4 Examen: con la capa semántica

```
python practica/sesion_2/04_examen.py --modo capa --rapido
```

→ Mismo modelo y mismas preguntas: 5 de 5.

Para elegir no basta con «parece que va bien»: hace falta un examen

1

Preguntas

Variadas y escritas por nosotros: también alguna imposible.

2

Respuesta correcta

Calculada a mano con SQL revisado y contrastada con el 03.

3

El modelo responde

Con su herramienta, como lo haría en el trabajo de verdad.

4

El código pone nota

¿Están todas las cifras correctas? ¿Dice que no a lo imposible?

Lo caro es el paso 2
Sin respuestas correctas no hay nota; sin nota, se elige por impresión.

Temperatura 0 y repeticiones
Un modelo fiable acierta siempre, no a veces.

Las respuestas correctas, calculadas a mano

QUÉ PONE A PRUEBA (13 PREGUNTAS)

- Neto de una tienda
- Neto de la empresa
- Pedidos y unidades
- Ticket por tienda
- Ranking de tiendas
- Un día concreto
- Un rango de fechas
- Producto más vendido
- Abonos de un periodo
- Stock
- Pedidos de 4+ unidades
- La línea histórica

Una imposible: clientes

EJEMPLO · P01 · VENTAS NETAS DE MALASAÑA EN A

verdad.py

```
-- Ventas: por la fecha de la venta
SELECT SUM(importe_linea) FROM ventas
WHERE local_id = 'MAD-MAL'
  AND fecha_venta BETWEEN '2026-03-02' AND '2026-03-29'
                                                  → 42.000
-- Abonos: por su fecha; la tienda, desde la venta
SELECT SUM(r.importe_reembolso) FROM reembolsos r
JOIN ventas v USING (linea_id) WHERE v.local_id = 'MAD-MAL'
  AND r.fecha_reconocimiento BETWEEN '2026-03-02' AND …
                                                  → 2.000
```

Neto A = 42.000 − 2.000 = **40.000 €** ✓ coincide con el 03

Cada respuesta, **escrita y revisada a mano**. Es lo más caro del examen… y lo que hace que valga.

PRÁCTICA 2 · 04_examen.py --ver

Tres exámenes, para no hacernos trampas

13

Batería

PARA DISEÑAR

Con ella probamos cada idea y la mejoramos.

10

Validación

PARA COMPROBAR

Preguntas nuevas: ¿la mejora vale para otras preguntas?

10

Prueba final

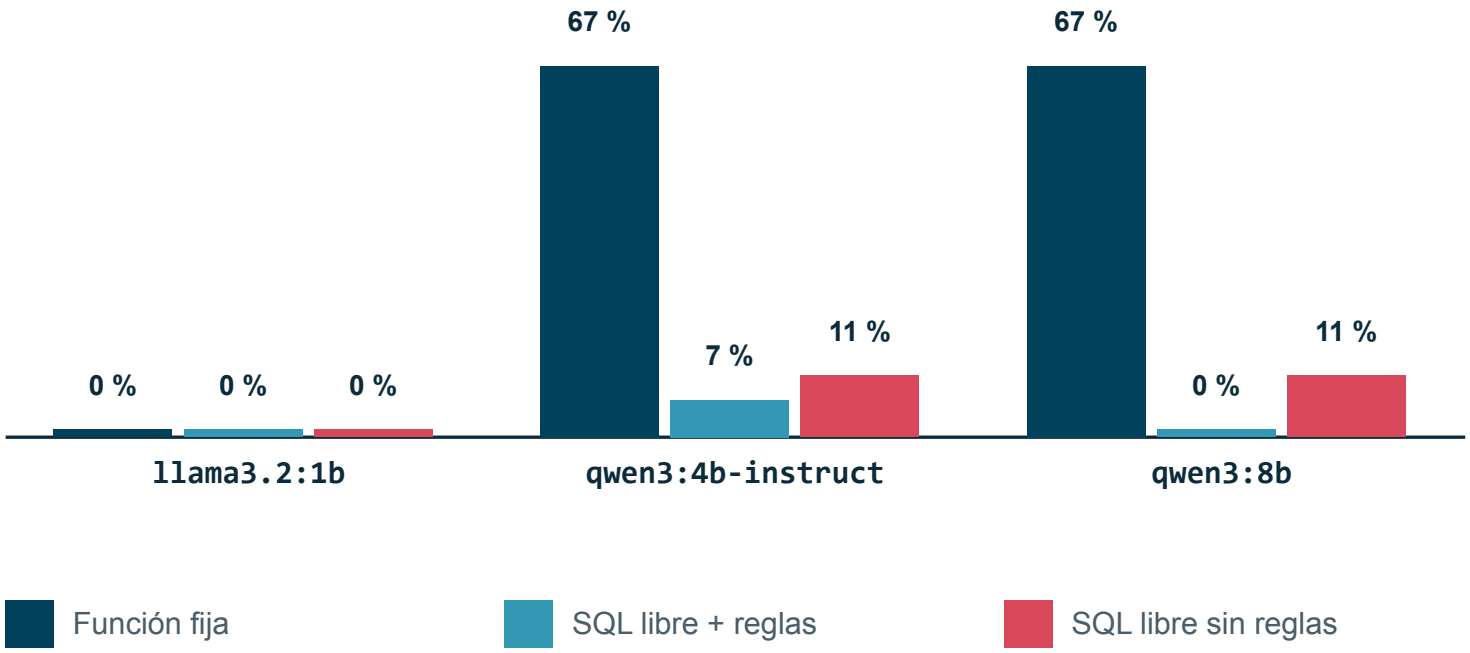
PARA DECIDIR

Escritas antes de la versión definitiva: el examen limpio.

Como estudiar con los ejercicios del libro y examinarse con **otros nuevos**: si solo miras la batería, te engañas.

No acierta el modelo más grande, sino la mejor herramienta

CAMPOS CORRECTOS · Q01 NETO · Q02 CESTA · Q03 HIST-JOIN-001 · 2 REPETICIONES



COSTE

`llama3.2:1b`
1,3 GB en disco · 1,7 GB cargado · 9 s por pregunta

`qwen3:4b-instruct`
2,5 GB en disco · 3,9 GB cargado · 25 s por pregunta

`qwen3:8b`
5,2 GB en disco · 6,3 GB cargado · 39 s por pregunta

La función acierta lo que cubre y no inventa lo que no cubre (Q03).

El resultado: de 58 % a 91 % de aciertos

ACIERTOS EN 33 PREGUNTAS (BATERÍA + VALIDACIÓN + PRUEBA FINAL) · SEGUNDOS POR PREGUNTA



La palanca grande es **cómo le damos los datos**. Elegimos el **30B con la capa**: 91 %, a unos 15 s por pregunta.

Lo que queda: errores de significado

EL VERIFICADOR CAZA ESTO

«El ticket medio de Goya fue…»

25.858 €

Ese número no sale de ningún resultado de SQL (era 258,33 €).
✓ El código se lo devuelve al modelo para que lo corrija.

PERO NO ESTO

«¿Cuántas líneas tuvo Goya en B?»

307 líneas

SUM(cantidad): son unidades. Las líneas son 240.
✗ El número sí sale del SQL: el verificador lo da por bueno.

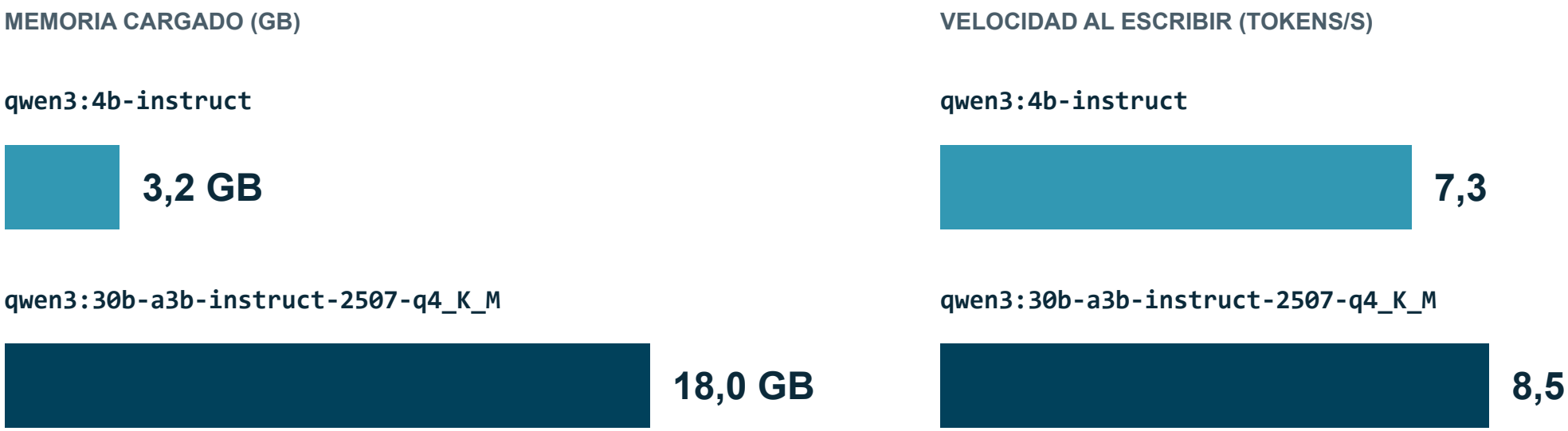
Como el «8 SKU» de la sesión 1: comprobar que una cifra existe no basta, hay que comprobar **a qué se refiere**. Por eso se enseña el SQL.

Hacen falta las dos cosas: la capa y un modelo capaz

	Tablas crudas + reglas	Capa semántica	
4B	61 %	67 %	**El 4B se pierde** Une vistas sin necesidad y suma unidades cuando se piden pedidos.
30B de expertos	58 %	91 %	**El 30B entiende la pregunta** Y, por ser de expertos, va tan rápido como el 4B.

La capa le quita al modelo **el negocio**; el modelo tiene que poner **la comprensión**.

Misma pregunta, dos modelos: lo que cuestan

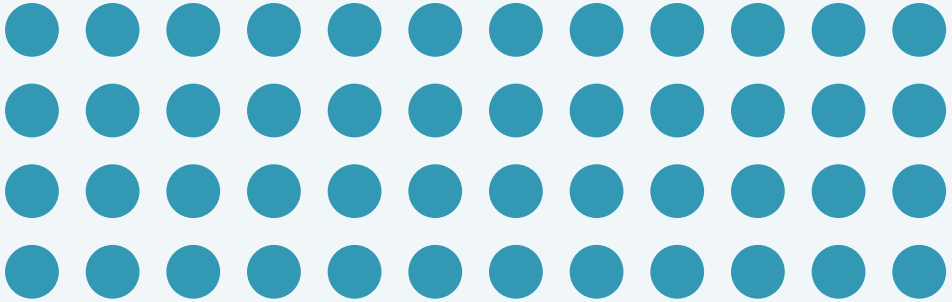


En disco: 2,5 GB el 4B y 18 GB el 30B. En la sesión 1, el 8B: 5,6 GB y unos 5 tokens/s.

El de 30B ocupa **casi seis veces más**… y escribe **más rápido** que el 4B. ¿Cómo puede ser?

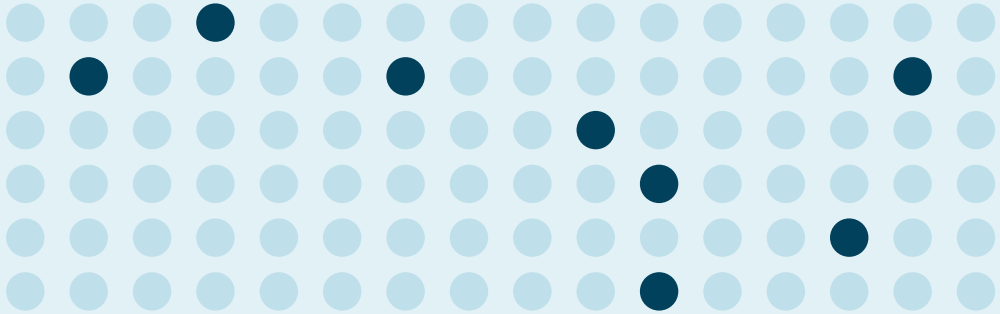
Cómo puede ser: un modelo «de expertos»

MODELO DENSO · qwen3:4b, qwen3:8b



Todo trabaja en cada palabra.
4B: 4.000 millones de parámetros, todos activos.

DE EXPERTOS · qwen3:30b-a3b



En cada palabra trabajan unos pocos expertos.
30.000 millones en memoria; unos 3.000 millones activos (8 de 128 expertos).

Cómo leer el nombre: **30b** = parámetros en total · **a3b** = activos por palabra («active 3B»).

La **memoria** la marca el total; la **velocidad**, los activos. Cabe en el Jetson y va rápido; en un portátil de 16 GB no cabe.

Qué elegimos, y por qué

Forma	Acierto	Qué se puede preguntar	Coste	Para qué
07 · Función fija	67 % (y no inventa)	Lo que cabe en la función	4B · 3,9 GB	Las preguntas de siempre
08 · SQL libre, tablas crudas	~60 %	Cualquier cosa	4B o 30B	Nada sin revisar
Capa semántica + 30B + verificador	**91 %**	**Cualquier cosa**	**18 GB · ~15 s**	**Explorar los datos**

Preguntas frecuentes, **función fija**. Explorar, **capa semántica + verificador**. Y siempre, **leer el SQL**.

Para dirección: cuatro frases, con cifras y límites

1

Hechos

Malasaña pasa de 40.000 a 26.000 € netos: −14.000 €, no 20.000.

2

Interpretación

Mantiene 120 pedidos, pero baja de 3,89 a 2,53 unidades por pedido.

3

Hipótesis

Los pedidos de 4 o más unidades caen de 108 a 7. Algo cambió el 30 de marzo.

4

Límites

No hay visitas, campañas aplicadas ni demanda no atendida.

La mejor respuesta no es la más rotunda, sino la más **exacta, trazable y honesta**.

PARA LLEVAR · BLOQUE I

Tres ideas para recordar

1 **El negocio, en el código.**
Con una capa semántica, el modelo pregunta lo que quiera y acierta 9 de cada 10.

2 **Para elegir, mide.**
Un examen con respuestas calculadas a mano, preguntas nuevas y una nota.

3 **Más grande no es la solución.**
Hace falta un modelo capaz y los datos bien presentados. Y siempre, leer el SQL.

Siguiente: los datos dicen cuánto y qué. Para el porqué, escuchamos una llamada y leemos los documentos.

EBIS Business Techschool

BLOQUE II · LA VOZ

La voz

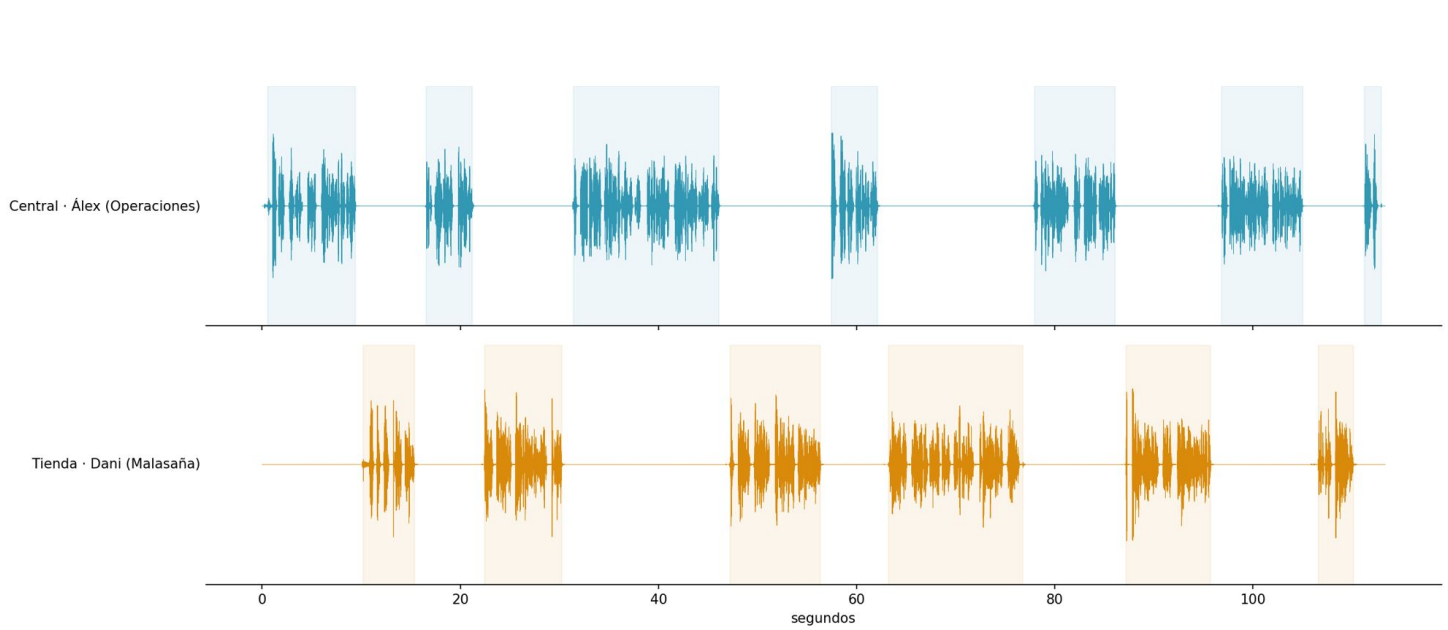
Una llamada de Operaciones a Malasaña: de la onda al texto, con Whisper en el Jetson.

Pregunta guía
¿Qué se dijo en la llamada, y cuánto nos podemos fiar del texto?

EN ESTE BLOQUE

1. **Qué es un audio**
   Números, ondas y frecuencias: cómo se representa el sonido.
2. **Whisper por dentro**
   Una matriz log-mel de 30 s, un codificador y un decodificador.
3. **Transcribir en local**
   Modelos, motores y por qué la GPU lo cambia todo.
4. **Dónde se equivoca**
   Y cómo mejorarlo: sin reentrenar y reentrenando.

La llamada: Operaciones llama a la tienda de Malasaña



113 s de conversación

2 canales: uno por persona

13 turnos en este montaje

16.000 números por segundo y canal

Álex
Central · Operaciones · canal izquierdo

Dani
Tienda · Malasaña · canal derecho

Queremos saber **qué se dijo**, **quién** lo dijo y **cuándo**, y contrastarlo con los datos.

EBIS Business Techschool

PRÁCTICA 3 · BLOQUE II

Ver la voz

Qué hay dentro de un audio y qué ve Whisper, sin ningún modelo todavía.

▶ Ahora, en la terminal

AL TERMINAR
Las slides siguientes explican cada vista.

PASOS

1 Qué hay en el archivo

```
python practica/sesion_2/06_ver_audio.py datos
```

→ Dos canales, 16.000 números por segundo, 16 bits: 3,6 millones de números.

2 De la palabra al número

```
python practica/sesion_2/06_ver_audio.py zoom
```

→ 2 s de palabras → 0,1 s de vibración → 5 ms: 80 números.

3 Lo que entra en Whisper

```
python practica/sesion_2/06_ver_audio.py mel
```

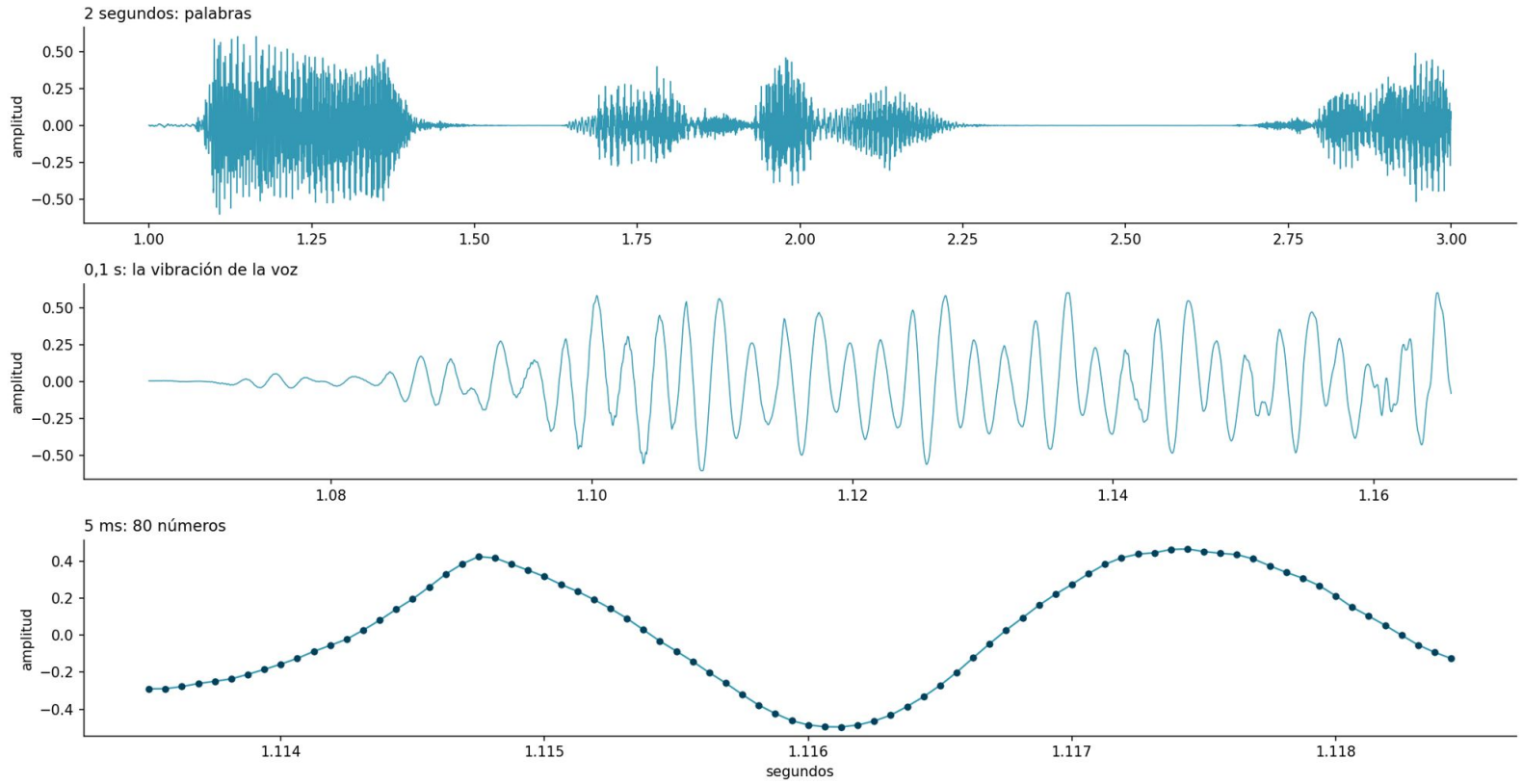
→ 30 s convertidos en características log-mel de 128 × 3000.

4 Todo a la vez, en directo

```
python practica/sesion_2/09_panel_en_directo.py
```

→ Reproduce el WAV y sincroniza onda, canales, espectrograma, log-mel y JSON de transcripción.

Un audio son números: 16.000 por segundo



16.000

números por segundo y canal

16 bits

cada número, de −32.768 a 32.767

3,6 M

números en toda la llamada

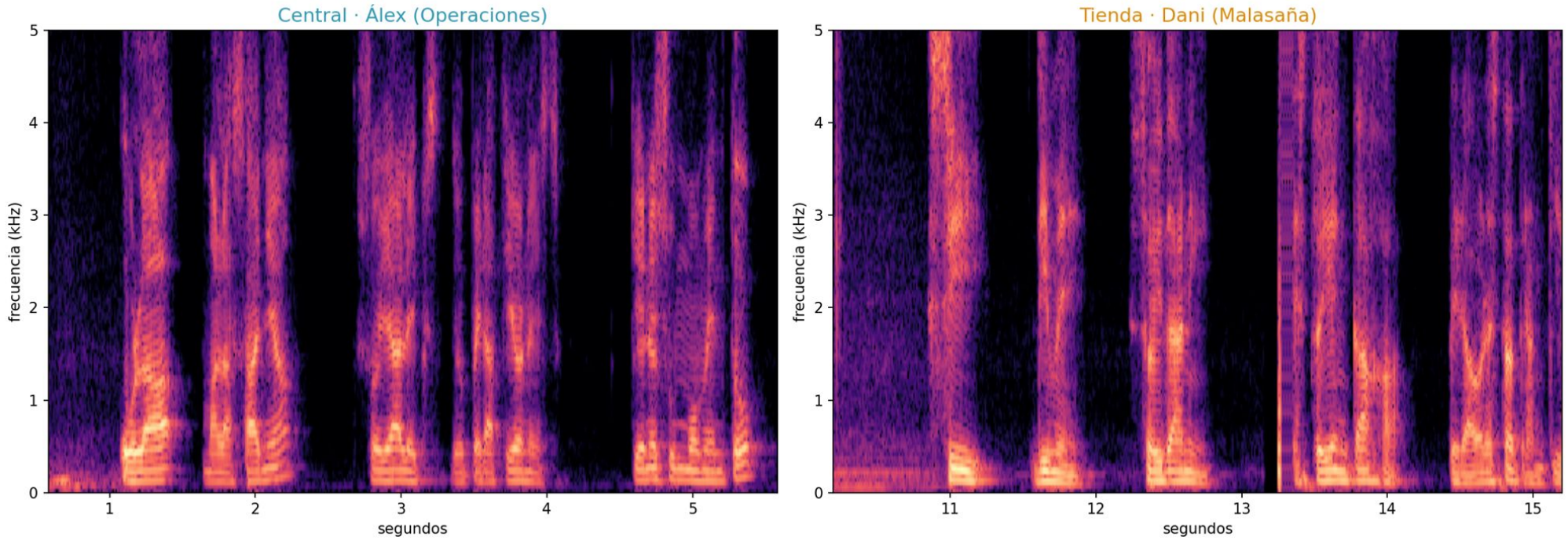
7,3 MB

en disco, sin comprimir

> **Un audio es una lista de números.**

PRÁCTICA 3 · 06_ver_audio.py zoom

La voz contiene muchas frecuencias: el espectrograma



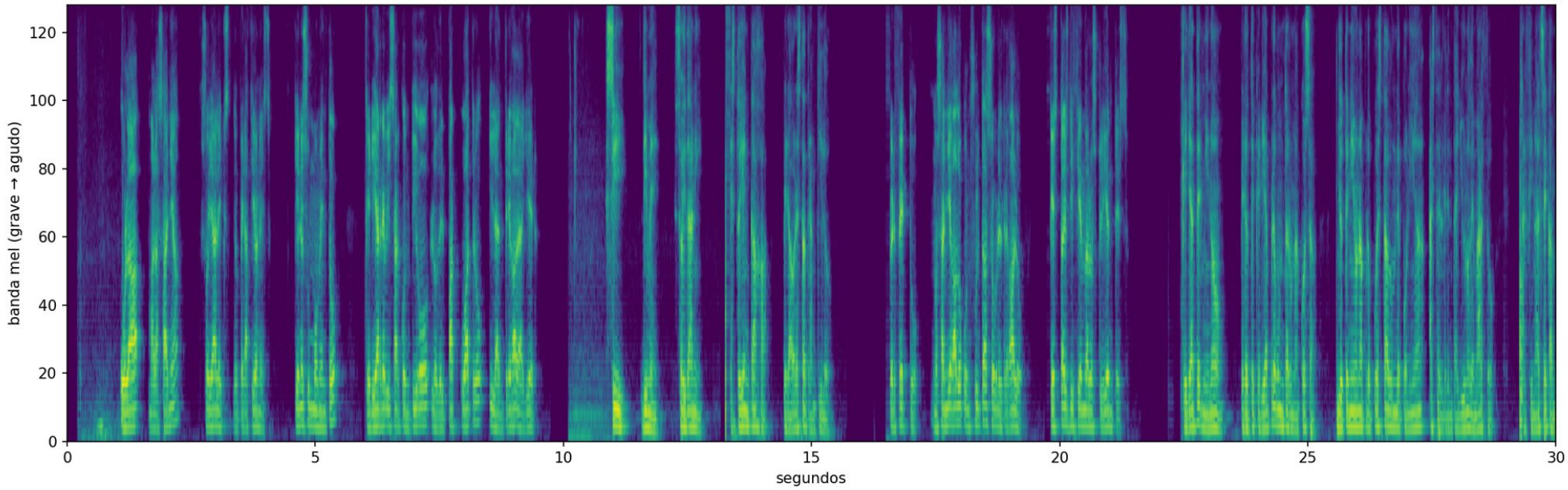
Eje vertical
frecuencia: grave abajo, agudo arriba

Color
energía relativa en cada banda

Eje horizontal
tiempo; patrones acústicos que cambian

PRÁCTICA 3 · 06_ver_audio.py espectrograma

Whisper recibe características acústicas numéricas



30 s

por trozo: Whisper procesa el audio en ventanas de 30 segundos

128 bandas

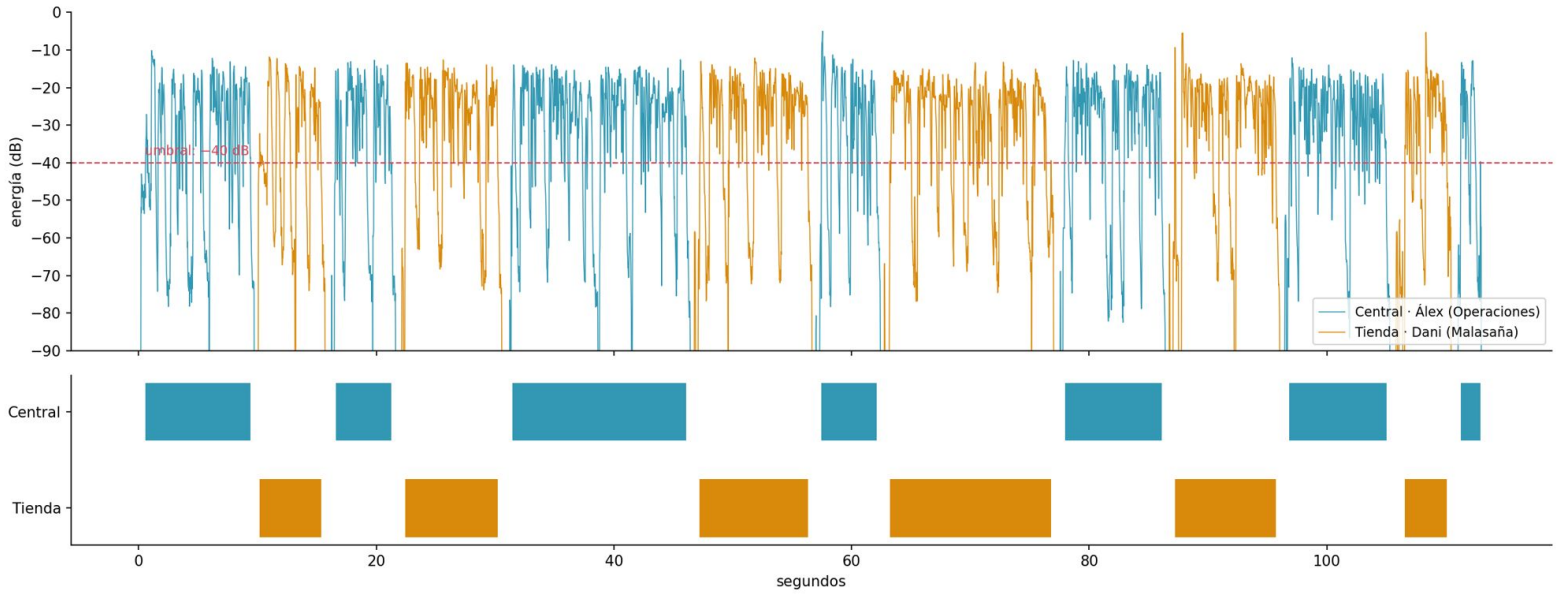
de frecuencia en escala «mel»: más detalle en graves, como el oído

3.000 columnas

una cada 10 ms: 100 por segundo

PRÁCTICA 3 · 06_ver_audio.py mel

Turnos de esta llamada: medir energía por canal



13 turnos

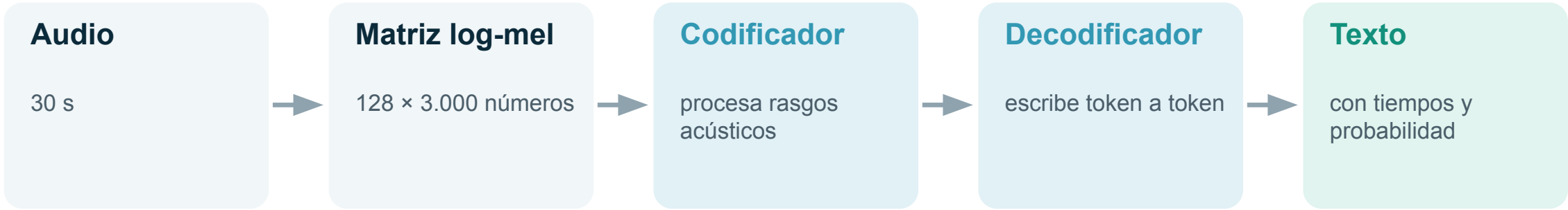
detectados en este audio con un umbral de energía

Whisper no sabe quién habla
El canal ya está asignado; no es un sistema general de diarización.

Por eso, grabar bien
vale más que un modelo más grande.

PRÁCTICA 3 · 06_ver_audio.py turnos

Whisper por dentro: procesa audio y escribe como un LLM



LO QUE ESCRIBE EL DECODIFICADOR (TOKENS)

```
<|startoftranscript|> <|es|> <|transcribe|> <|0.00|> Hola, Malasaña. Soy Álex, de Operaciones. <|9.40|> …
```

Unos tokens le dicen qué hacer
idioma, transcribir o traducir al inglés, tiempos y «aquí no hay voz».

Escribe como un LLM…
…así que también puede inventar. Lo veremos.

Cómo se entrenó: cientos de miles de horas de internet

2022

Whisper

680.000 h de audio con su transcripción, de internet. 96 idiomas además del inglés.

2022

large-v2

Los mismos datos, más entrenamiento.

2023

large-v3

1 M de horas + 4 M etiquetadas automáticamente por large-v2. 128 bandas mel.

2024

turbo

large-v3 con el decodificador recortado de 32 a 4 capas: casi igual y mucho más rápido.

Supervisión débil
Transcripciones de internet que nadie revisó una a una: la fuerza está en la escala y la variedad.

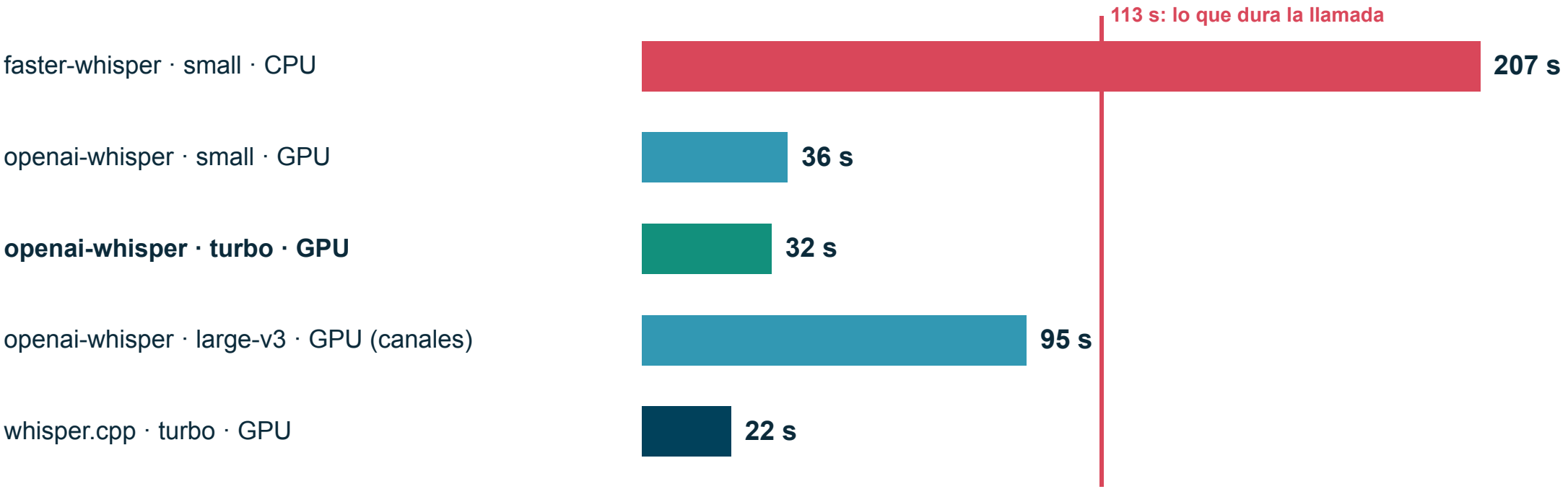
Abierto (licencia MIT)
Pesos y código descargables: corre en local y los datos no salen del equipo.

Elegir modelo: tamaño, velocidad y resultados observados

Modelo	Parámetros	Descarga	Llamada en el Jetson (GPU)	Lo que vimos
tiny · base	39 M · 74 M	75 · 140 MB	—	Para pruebas rápidas
small	244 M	0,5 GB	36 s	«PAK4», «Danny»
medium	769 M	1,5 GB	—	—
turbo	**809 M**	**1,6 GB**	**32 s (mezcla) · 45 s (canales)**	**Equilibrio de velocidad y calidad**
large-v3	1.550 M	2,9 GB	95 s (canales)	Más grande; también puede inventar y fallar

Se elige **el más pequeño que haga bien la tarea**, igual que con los LLM.

Tres motores, misma llamada: la GPU lo cambia todo



openai-whisper
PyTorch para Jetson + pip. Probabilidad por palabra. **El que usamos.**

faster-whisper
pip y listo, pero en el Jetson solo CPU. **Bueno para portátiles.**

whisper.cpp
El «primo» de llama.cpp. El más rápido; **hay que compilarlo** (25 min).

Voz abierta: Whisper y su familia

Whisper

OPENAI · MIT

El modelo de referencia: 100 idiomas, puntuación y tiempos.

faster-whisper

CTRANSLATE2

El mismo modelo, cuantizado y rápido en CPU.

whisper.cpp

GGML

En C/C++, como llama.cpp (el motor de Ollama). CPU y GPU.

distil-whisper

HUGGING FACE

Un alumno pequeño que imita al grande: destilación.

Parakeet · Canary

NVIDIA

Otros modelos abiertos de voz a texto.

Piper · Kokoro

TEXTO → VOZ

El camino de vuelta: que el sistema hable.

Ollama no ejecuta Whisper: para la voz hace falta otro motor.

EBIS Business Techschool

PRÁCTICA 4 · BLOQUE II

Transcribir con Whisper

La misma llamada, transcrita de varias formas: ¿qué cambia el texto?

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes explican los errores y cómo evitarlos.

PASOS

1 La mezcla, sin separar voces

```
python practica/sesion_2/07_transcribir.py --modo mono
```

→ En esta llamada: «ya nos entregaba»; es una salida dudosa, no un efecto aislado de la mezcla.

2 Un canal por persona

```
python practica/sesion_2/07_transcribir.py
```

→ La salida cambia a «ya no se entregaba». Revisamos el instante en el panel: Dani está en silencio.

3 El modelo más grande

```
python practica/sesion_2/07_transcribir.py --modelo large-v3 --solo-voz
```

→ ¿Qué dice Dani en el segundo 10,4? Repite con --solo-voz.

4 Verla y escucharla

```
python practica/sesion_2/09_panel_en_directo.py
```

→ Panel en directo: llamada, canales, espectrograma y transcripción sincronizados.

Una frase ambigua: mono y canal dan hipótesis distintas

MEZCLA MONO · TURBO

*«Desde el 30 ya **nos** entregaba la tote bag»*

`p(«nos») = 0,626 · dudosa`

⚠ **Salida a comprobar**

CANAL CENTRAL · TURBO

*«Desde el 30 ya **no se** entregaba la tote bag»*

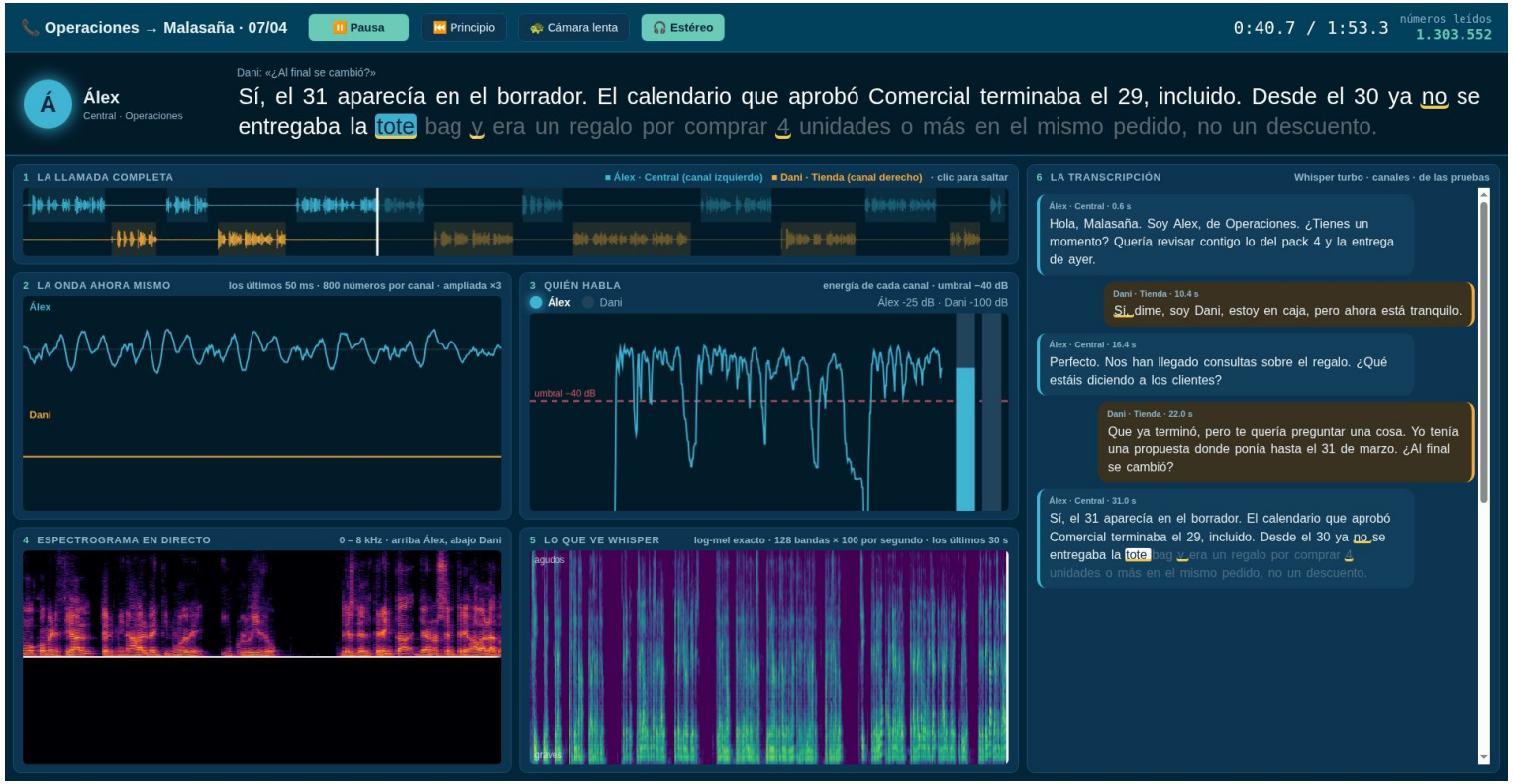
`p(«no») = 0,655 · dudosa`

⚠ **También requiere escucha**

Habla Álex; Dani está en silencio. La diferencia no prueba que los canales causaran el cambio.

PRÁCTICA 4 · 07_transcribir.py --modo mono

La puntuación por palabra: una alerta para revisar



PALABRAS DUDOSAS (P < 0,70)

se iba	`0,48 · 0,65` era «siguió»
nos	`0,63` era «no se»
tote	`0,58` correcta, pero dudosa
pack	`0,07` escrita por la pista, no oída

No es infalible
También hay errores con p alta: se verifica igual.

PRÁCTICA 4 · 09_panel_en_directo.py

A veces Whisper escribe incluso cuando apenas hay voz

LARGE-V3 · CANAL DE DANI · 10,4 s

«¿Qué pasa?»

Ahí solo hay una respiración. El decodificador, que escribe como un LLM, completa con algo que «suena» a llamada.

CON --SOLO-VOZ

(nada)

Whisper solo escucha los tramos con energía (los 13 turnos). Sin silencios que rellenar, no hay invento.

Un clásico en español: «Subtítulos realizados por la comunidad de Amara.org»
Sale de los subtítulos con los que se entrenó: aparece en audios en silencio o con música.

PRÁCTICA 4 · 07_transcribir.py --modelo large-v3

Más grande no siempre acierta más

En nuestras pruebas	small · canales	turbo · solo voz	large-v3 · solo voz
«Pack 4»	PAK4	Pack 4	PAC-4
La tienda … abierta	se abierta	siguió abierta	seguía abierta
Frase espuria en silencio	—	—	— (con solo voz)
Nota de lectura	modelo menor	rápido y buen ejemplo	mayor; error distinto

Comparación cualitativa: modelo y preparación producen hipótesis distintas. No hay WER sin referencia humana.

Cómo reducir errores sin reentrenar

1	**Un canal por persona**	Conserva hablante/pista; ayuda ante solapamiento	**Este ejemplo no prueba causalidad**
2	**Solo voz (umbral de energía)**	Recorta actividad baja; puede cortar voz débil	**«¿Qué pasa?» desaparece aquí**
3	**Pista de contexto**	Sugiere vocabulario; no cambia audio/pesos	**«PAK4» → «pack 4»; revisar**
4	**Elegir modelo**	Calidad frente a tiempo	**turbo: el equilibrio**
5	**Contrastar con datos**	Detecta errores semánticos	**Caja / calendario / stock**

La pista orienta el vocabulario, no corrige la señal: verificar términos sugeridos.

PRÁCTICA 4 · 07_transcribir.py --pista

Y si no basta: adaptar Whisper a un dominio

1

Preparar

Audio + texto humano revisado; acordar formato de importes, nombres y dudas.

2

Separar

Train / validación / test por llamada/hablante; test intacto.

3

Ajustar

Desde Whisper. LoRA congela la base y aprende adaptadores.

4

Evaluar

WER + errores críticos en llamadas humanas no vistas; comparar base/adaptado.

CÓMO SE MIDE: WER (WORD ERROR RATE)

WER = (sustituciones + borrados + inserciones) / palabras

LoRA, en una imagen mental
Deja quietas las perillas base; añade pequeños adaptadores. Menos memoria/almacenamiento, no menos necesidad de validar.

Del audio al dato: lo que dice la llamada, contrastado

Lo que dice la llamada	Dónde se comprueba	
Día 6: sin Street One negro y arena, 41 y 42, de 10 a 16 h	disponibilidad.csv: 4 SKU, 10:00–16:00	✓ coincide
El calendario aprobado terminaba el 29; el borrador decía 31	Documentos: calendario y propuesta	→ bloque III
«Avería del vehículo»	Informe de logística (PDF)	→ bloque III
Regalo por 4 o más unidades en el mismo pedido	ventas.csv: 108 → 7 pedidos de 4+	✓ coincide
«No tengo un número de visitas»	No existe en ningún dato	límite

La transcripción es **una fuente más**: se cita con quién y cuándo… y se contrasta.

PARA LLEVAR · BLOQUE II

Tres ideas para recordar

1 **Whisper procesa rasgos del audio y escribe como un LLM.**
Convierte el sonido en características numéricas y aun así puede inventar.

2 **Preparar la entrada puede ayudar más que aumentar el modelo.**
Solo-voz quitó una frase espuria observada; el caso mono/canal produjo hipótesis distintas que igual hubo que revisar.

3 **El texto es una fuente: se cita y se contrasta.**
Con quién lo dijo, cuándo y con qué probabilidad.

Siguiente: buscar en los documentos lo que dice la llamada, y citarlo. RAG.

EBIS Business Techschool

BLOQUE III · PREGUNTAR A LOS DOCUMENTOS

Preguntar a los documentos

Un RAG local: buscar primero, responder después y citar de dónde sale cada cosa.

Pregunta guía
¿Qué dicen los documentos y la llamada, y dónde lo dicen?

EN ESTE BLOQUE

1. **Qué tenemos**
   Diez fuentes, una historia en fechas y cuatro trampas.
2. **Preparar las fuentes**
   Trocear por su estructura y etiquetar: la ficha de cada fragmento.
3. **Buscar bien**
   Embeddings, palabras clave y búsqueda híbrida, medidos.
4. **Responder con fuentes**
   Citas, versiones y un código que revisa al modelo.

El modelo no ha leído nuestros documentos: hay que dárselos

PEGARLO TODO EN EL PROMPT

- Nuestro corpus: unas 6.000 palabras (~9.000 tokens): cabría.
- Con mil documentos, no cabe: el contexto es memoria (sesión 1).
- El modelo se pierde entre lo que no importa.
- Difícil saber de dónde sale cada frase.

BUSCAR Y LUEGO RESPONDER (RAG)

- Solo los 5 fragmentos relevantes: ~2.000 tokens.
- Escala a miles de documentos.
- Cada fragmento llega con su ficha: documento, página, fecha y estado.
- Cada afirmación, con su cita comprobable.

RAG = Retrieval-Augmented Generation: buscar primero, responder después… **citando**.

Diez fuentes, una historia: qué tenemos para preguntar

DECIDIDO
qué se aprobó

Calendario aprobado
MD · aprobado
`calendario_comercial_aprobado.md`
Pack 4: del 02/03 al 29/03.

Propuesta Pack 4
MD · sustituido
`propuesta_pack4_borrador.md`
Proponía hasta el 31/03. Ya no manda.

OCURRIDO
lo que pasó y alguien contó

Diario de tienda
MD · registro
`diario_operaciones_abril.md`
Cartel fuera el 30/03; roturas 6 y 16/04.

Informe de logística
PDF · emitido
`informe_logistica_abril.pdf`
Avería el 6/04; reposición a las 16:00.

La llamada del 07/04
AUDIO → TEXTO · automática
`Whisper turbo, solo voz`
«Terminaba el 29», el día 6, sin visitas.

PREVISTO
planes y avisos

Aviso de obras
MD · recibido
`aviso_obras.md`
Obras programadas del 01/04 al 20/04.

Programado no es ocurrido
El aviso dice cuándo estaba previsto. Nada confirma que empezaran ese día.

REGLAS
cómo leer los datos

Diccionario de datos
MD · vigente
`diccionario_datos.md`
Cada columna, y qué datos no existen.

Política de reembolsos
MD · vigente
`politica_reembolsos.md`
Un abono cuenta el día que se reconoce.

Manual de datos
PDF · vigente
`manual_datos_y_criterios.pdf`
Métricas y reglas; pág. 7: la ficha.

Expediente comercial y de operaciones
PDF de 8 páginas: el encargo, copias de 6 documentos y lo pendiente

pág. 1 Encargo | pág. 2 Calendario | pág. 3 Diario | pág. 4 Logística | pág. 5 Stock | pág. 6 Obras | pág. 7 Borrador | pág. 8 Pendientes

La historia en fechas: cada fuente, un momento distinto



Cuándo se **escribe**, qué se **planifica** y qué **pasa**: tres fechas que no son intercambiables.

Cuatro trampas escondidas en las fuentes

1 Dos versiones casi iguales

Borrador «hasta el 31/03» · aprobado «hasta el 29/03»

Para la búsqueda son casi el mismo texto.

✓ **La ficha dice cuál está sustituido.**

2 Fechas con significados distintos

25/03 recibido · 01/04 programado · 20/04 previsto

Ninguna dice que las obras empezaran.

✓ **Conservar el significado de cada fecha.**

3 Lo mismo, en dos sitios

El expediente copia 6 documentos (Markdown y PDF)

Un hecho repetido no son dos pruebas.

✓ **Citar la fuente; no sumar copias.**

4 Una fuente escrita por una máquina

La llamada: Whisper produce «nos» / «no se» en salidas distintas

Frase ambigua; no prueba que los canales causen la diferencia.

✓ **Estado «transcripción automática» y escuchar/contrastar.**

Y lo que **no está** (visitas, regalos entregados, margen): la respuesta correcta es «**no consta**».

Un RAG en cuatro pasos, todo en el Jetson

1

Trocear y etiquetar

10 fuentes → 51 fragmentos con su ficha

`10_corpus.py`

2

Índice vectorial

cada fragmento → 1.024 números (bge-m3), en DuckDB

`11_indice.py`

3

Buscar

la pregunta → los 5 fragmentos más relevantes

`12_buscar.py`

4

Responder y verificar

solo con esos fragmentos, citando [F#]

`13_preguntar_documentos.py`

LAS FUENTES (Y NUNCA LA CARPETA DEL PROFESOR)

6 documentos Markdown

3 PDF (incluidos expediente y manual)

La llamada (Whisper turbo, solo voz)

Embeddings y modelo en Ollama, índice en un archivo DuckDB: nada sale del equipo.

Las mismas piezas, en el Jetson y en la nube

	ARCHIVOS	TROCEAR Y ETIQUETAR	EMBEDDINGS	ÍNDICE	BUSCAR	RESPONDER
EN EL JETSON lo de hoy	**data/documentos/** carpeta local	**10_corpus.py** pymupdf + reglas	**bge-m3** en Ollama	**DuckDB** un archivo: ficha + vector	**12_buscar.py** híbrida (RRF)	**qwen3 30B** en Ollama
EN AZURE lo equivalente	**Blob Storage** o Data Lake	**Function o skillset** + Document Intelligence si hay escaneados	**Azure OpenAI** text-embedding-3	**Azure AI Search** campos filtrables + vector	**AI Search** consulta híbrida + filtros	**Azure OpenAI** GPT, con las citas

La ficha va con el vector
Se prepara al trocear y se guarda en el índice, al lado del vector. En AI Search: filtrar estado = vigente.

¿MongoDB, Cosmos DB, PostgreSQL?
También guardan vectores (Atlas Vector Search, pgvector). Lo importante: vector + ficha + filtros.

Nada sale de DropLab (manual, pág. 8)
En la nube: suscripción propia, red privada, región UE. Es cumplimiento, no solo técnica.

EBIS Business Techschool

PRÁCTICA 5 · BLOQUE III

Preparar y buscar

De diez fuentes a un índice que encuentra el fragmento justo, con su ficha.

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes explican el troceado, los embeddings y la búsqueda.

PASOS

1 Trocear y etiquetar las fuentes

```
python practica/sesion_2/10_corpus.py
```

→ 10 fuentes → 51 fragmentos: un ■ por fragmento, del color de su estado.

2 Ver cómo se corta un documento

```
python practica/sesion_2/10_corpus.py --trocear calendario
```

→ Antes y después: qué va a la ficha, qué se quita y dónde se corta.

3 Todo el corpus, en el navegador

```
python practica/sesion_2/10_corpus.py --html --abrir
```

→ Original y fragmentos lado a lado: pasa el ratón por un fragmento.

4 Crear el índice vectorial

```
python practica/sesion_2/11_indice.py
```

→ 51 vectores de 1.024 números (bge-m3) en DuckDB, en unos 15 s.

5 Buscar

```
python practica/sesion_2/12_buscar.py "¿Hasta qué día estuvo vigente el Pack 4?" --explicar
```

→ El primero es… el borrador. Y el buscador añade el vigente.

Así se trocea un documento: por secciones, con su ficha

calendario_comercial_aprobado.md

✂ cabecera → la ficha (no es un fragmento)

```
# DropLab · Calendario comercial aprobado                    → título
Documento CAM-2026-03-FINAL · Versión 2 · Estado: APROBADO    → código, estado
Aprobación: 01/03/2026 · Responsable: dirección comercial     → fecha, responsable
```

✂ sección «Campañas» → fragmento #2

```
## Campañas
| Tienda | Campaña | Inicio | Último día incluido |...
| MAD-MAL | Pack 4 | 02/03/2026 | 29/03/2026 | Tote bag...
| MAD-GOY | Semana Running | 09/03/2026 | 15/03/2026...
| BCN-EIX | Sin campaña local | 02/03/2026 | 26/04/2026...
```

✂ sección «Pack 4: ejecución» → fragmento #3

```
## Pack 4: ejecución
Incluye zapatillas, camisetas y sudaderas. Cada par...
Pack 4 no modifica los precios de lista ni añade...
Este calendario sustituye a CAM-2026-03-DRAFT...   → DRAFT: sustituido
Operaciones no incluyó en la exportación el registro...
```

#2 calendario_comercial_aprobado.md · Campañas

CAM-2026-03-FINAL | aprobado | 01/03/2026 | dirección comercial

TEXTO QUE SE INDEXA

Calendario comercial aprobado (CAM-2026-03-FINAL, aprobado, 01/03/2026) · Campañas

```
| MAD-MAL | Pack 4 | 02/03/2026 | 29/03/2026 | Tote bag...
```

#3 calendario_comercial_aprobado.md · Pack 4: ejecución

CAM-2026-03-FINAL | aprobado | 01/03/2026 | dirección comercial

TEXTO QUE SE INDEXA

Calendario comercial aprobado (CAM-2026-03-FINAL, aprobado, 01/03/2026) · Pack 4: ejecución

```
Incluye zapatillas, camisetas y sudaderas. Cada par...
```

La cabecera **azul** (de la ficha) se pega delante del texto: el fragmento se entiende aunque llegue solo.

PRÁCTICA 5 · 10_corpus.py --trocear calendario

Cada fuente se corta por su estructura natural

MARKDOWN

6 archivos → 21 fragmentos

1 sección (##) = 1 fragmento

✂ cabecera → ficha

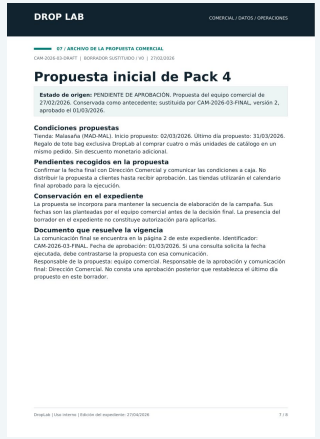
✂ ## Campañas → #2

✂ ## Pack 4: ejecución → #3

PDF

3 archivos → 17 fragmentos

1 página = 1 fragmento


DROP LAB

Propuesta inicial de Pack 4

✗ logotipo, pie, n.º de pág.

→ ficha: CAM-2026-03-DRAFT · BORRADOR SUSTITUIDO · 27/02

→ se unen las frases cortadas

! las tablas salen celda a celda

expediente · pág. 7: el borrador

LLAMADA

13 turnos → 13 fragmentos

2 turnos seguidos, con solape



■ Álex ■ Dani · cada turno sale en dos fragmentos: la respuesta, con su pregunta.

Usamos los delimitadores que reconocemos en estas fuentes; revisamos los cortes, no es una regla universal.

PRÁCTICA 5 · 10_corpus.py --html --abrir

Cada fragmento lleva su ficha: lo exige el propio manual

10_corpus.py --ver 36

```
"cita":     "propuesta_pack4_borrador.md · Propuesta"
"titulo":   "Propuesta de campaña Pack 4"
"codigo":   "CAM-2026-03-DRAFT"
"fecha":    "27/02/2026"
"responsable": "equipo comercial"
"estado_documento": "borrador"
"estado":   "sustituido"
"sustituido_por": "CAM-2026-03-FINAL"

"texto": "Tienda: MAD-MAL. Inicio propuesto: 02/03/2026.
          Último día propuesto: 31/03/2026. …"
```

MANUAL DE DATOS · PÁG. 7

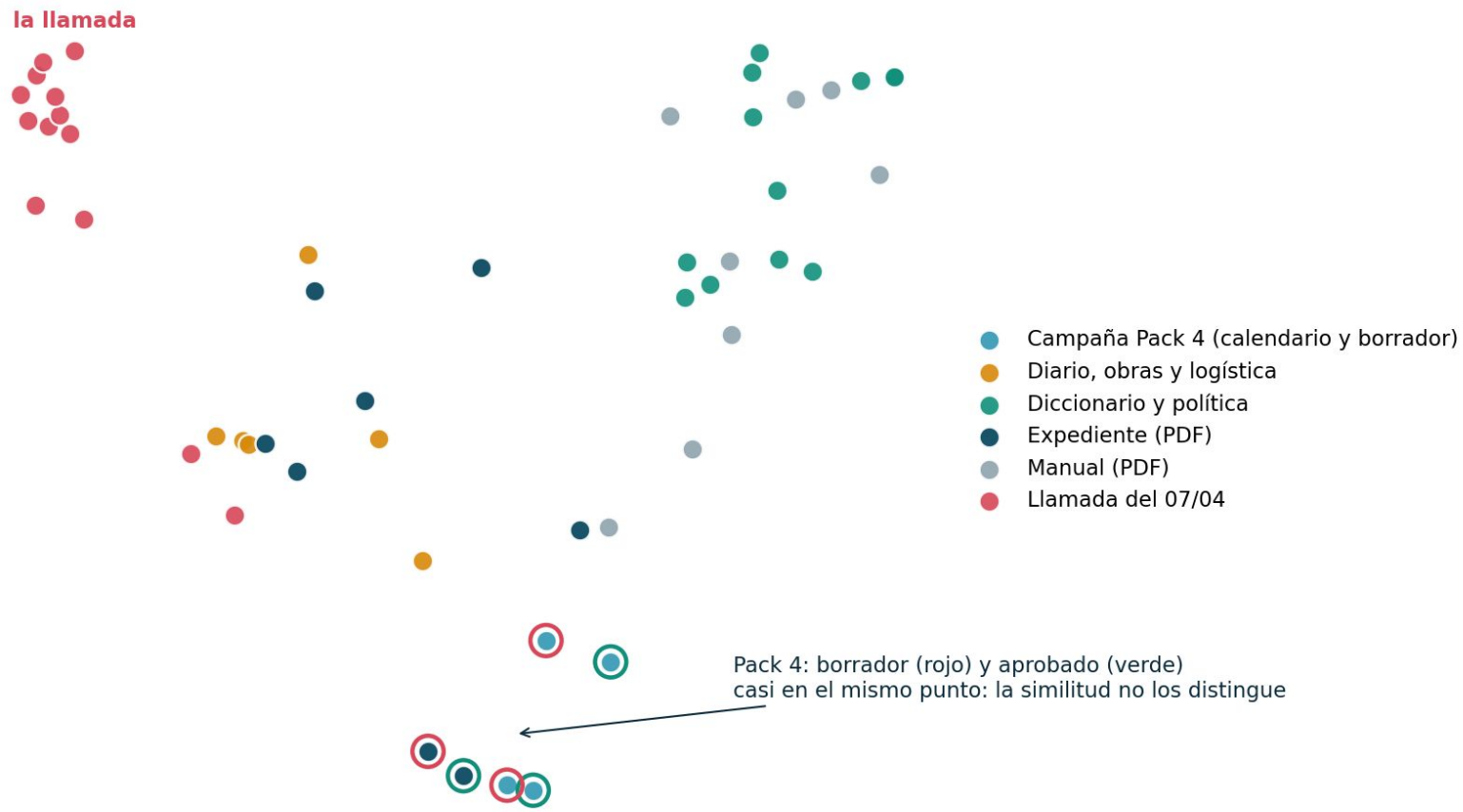
«Todo fragmento utilizado en un informe debe conservar el documento de origen, sección o página, fecha, responsable y estado documental.»

«Sustituido»: regla literal en Python

El código busca «sustituye a CAM-2026-03-DRAFT», conserva la cita donde lo encontró y actualiza el estado de las copias del borrador. Es una inferencia por regla para este caso, no una validación general de versiones; el embedding no la decide.

PRÁCTICA 5 · 10_corpus.py --ver 36

Embeddings: el significado convertido en números



SIMILITUD COSENO · BGE-M3

«avería del vehículo»
↔ «el camión se estropeó»

0,67

«tote bag»
↔ «bolsa de regalo»

0,67

«tote bag»
↔ «HIST-JOIN-001»

0,39

Sin una palabra en común, y cerca.

El problema
Borrador y aprobado dicen casi lo mismo: quedan juntos.

PRÁCTICA 5 · 11_indice.py --comparar

Tres formas de buscar

Semántica

por significado (embeddings)

✓ **Entiende «vigencia» ≈ «último día»**

✗ **Puede perder códigos raros**

Palabras (BM25)

por palabras clave, como un buscador clásico

✓ **Clava «HIST-JOIN-001»**

✗ **No entiende sinónimos**

Híbrida

las dos, unidas por su posición (RRF)

✓ **Lo mejor de las dos**

✗ **Un poco más lenta**

Con «¿hasta cuándo duró el Pack 4?» falla la de palabras; con «HIST-JOIN-001», la semántica duda.

La trampa de las versiones: lo más parecido es el borrador

«¿HASTA QUÉ DÍA ESTUVO VIGENTE EL PACK 4?» · BÚSQUEDA HÍBRIDA

1	expediente · pág. 7 · Propuesta inicial	sustituido · 31/03
2	borrador.md · Pendientes	sustituido
3	expediente · pág. 2 · Calendario aprobado	aprobado · 29/03
4	borrador.md · Propuesta	sustituido · 31/03
5	calendario aprobado.md · Pack 4: ejecución	aprobado
+	calendario aprobado.md · Campañas	añadido: el vigente

TRES DEFENSAS

Estado en la ficha
«sustituido por CAM-2026-03-FINAL»

El código añade el vigente
si sale un sustituido, trae el que lo sustituye

Una regla para el modelo
manda el aprobado; el borrador, solo como antecedente

La similitud no sabe **qué documento manda**. Los **metadatos**, sí.

PRÁCTICA 5 · 12_buscar.py --explicar

EBIS Business Techschool

PRÁCTICA 6 · BLOQUE III

Responder con fuentes

Respuestas que citan cada afirmación, verificadas por el código, y todo junto.

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes: qué verifica el código, qué no y la nota.

PASOS

1 La trampa de las versiones

```
python practica/sesion_2/13_preguntar_documentos.py "¿Hasta qué día estuvo vigente el Pack 4?"
```

→ 29/03 del aprobado; el 31/03 del borrador, solo como antecedente.

2 Una pregunta que da algo por hecho

```
python practica/sesion_2/13_preguntar_documentos.py "¿Cuándo empezaron las obras?"
```

→ ¿Dice «empezaron» o «estaba previsto»? Léelo contra la fuente.

3 El examen

```
python practica/sesion_2/14_evaluar_rag.py -k 3
```

→ 4 modelos de embeddings × 3 formas de buscar: ¿la fuente, entre los 3 primeros?

4 Todo junto: datos + documentos

```
python practica/sesion_2/15_analista.py
```

→ Cifras con SQL [S#] y hechos con fuente [D#], en una respuesta.

Medido: ¿encuentra la fuente correcta?

Modelo de embeddings	Semántica	Palabras	Híbrida
bge-m3	**16/16**	**14/16**	**16/16**
qwen3-embedding:0.6b	16/16	14/16	16/16
embeddinggemma	13/16	14/16	15/16
nomic-embed-text	13/16	14/16	15/16

¿ESTÁ LA FUENTE ESPERADA ENTRE LOS 3 PRIMEROS FRAGMENTOS? · 16 PREGUNTAS

Solo palabras: 14/16
falla «vigencia del Pack 4» y la pregunta sobre la llamada

nomic: 13/16
pensado para inglés: peor en español

Híbrida: rescata
a los modelos flojos. Elegimos bge-m3 híbrida

PRÁCTICA 6 · 14_evaluar_rag.py -k 3

Una respuesta con fuentes: cada afirmación, con su cita

*«El Pack 4 estuvo vigente desde el 02/03/2026 hasta el 29/03/2026, ambos incluidos **[F3]**. A partir del 30/03/2026 se retiró el regalo **[F3]**. Esta vigencia fue aprobada y comunicada el 01/03/2026 **[F3]**. La propuesta inicial que indicaba hasta el 31/03/2026 fue sustituida **[F1]**.»*

Cita	Documento	Dónde	Estado	Fecha
[F3]	Calendario comercial aprobado	expediente · pág. 2	aprobado	01/03/2026
[F1]	Propuesta inicial de Pack 4	expediente · pág. 7	sustituido	27/02/2026

✓ **Verificación del código: todas las frases citan, las citas existen y cada fecha aparece en su fuente.**

El código revisa al modelo

✓ **Las citas existen**
Si cita [F9] y solo hay 6 fragmentos, avisa.

✓ **Cada frase lleva cita**
«Sin cita, no se afirma» (salvo «no consta»).

✓ **Cada cifra está en su fuente**
Si dice «30/03/2026» y [F1] no lo contiene, avisa.

✓ **Un borrador no es lo vigente**
Si cita un sustituido sin decir que es borrador, avisa.

Es el mismo verificador del bloque I, ahora para **fuentes**: una cifra sin respaldo, **avisa**.

Lo que no puede comprobar: si la frase dice lo mismo que la fuente. Siguiente slide.

Lo que la verificación no ve: la cita es real, el significado no

«¿CUÁNDO EMPEZARON LAS OBRAS?»

«…el inicio efectivo fue el 1 de abril [F2]»

La fuente dice «inicio programado» y que no hay confirmación de la ejecución. La pregunta lo daba por hecho.

«¿QUIÉN LLAMÓ A LA TIENDA Y PARA QUÉ?»

«…se realizó el mismo día que el aviso de obras fue recibido [F1][F5]»

El aviso se recibió el 25/03 y la llamada es del 07/04. Inventa una relación entre fuentes, con cita.

Como el «8 SKU» y las «307 líneas»: comprobar que una cita existe no basta; hay que **leerla**.

Medido: la nota de las respuestas

Modelo	Hechos correctos	Cita la fuente esperada	Verificación limpia	Por pregunta
qwen3:30b-a3b-instruct	**16/16**	**16/16**	**16/16**	**24 s**
qwen3:4b-instruct	16/16	15/16	10/16	35 s

La imposible, bien
«¿Cuántas tote bags se entregaron?» → «No consta en los documentos. Faltaría el registro de regalos…»

Pero la nota no lo ve todo
El examen mira hechos y fuentes; los errores de significado de la slide anterior pasaron.

Bien buscado y bien citado, el modelo acierta. Y aun así, **una persona lee las fuentes**.

PRÁCTICA 6 · 14_evaluar_rag.py --parte respuesta

Sin consultar, se lo inventa todo… y el código lo para

LO QUE ESCRIBIÓ, SIN CONSULTAR NADA

«…las ventas netas bajaron un 23 % [S1]… los abonos aumentaron un 45 % [S3]. El 28 de marzo se registró un error en el sistema de precios que afectó a 12 productos… [D1]»

Ni el error de precios ni esas cifras existen. Cita [S1] y [D1]… sin haber hecho ninguna consulta.

LO QUE HIZO EL CÓDIGO

0 fuentes

en la lista: las citas no corresponden a nada consultado.

6 cifras sin respaldo

12, 15, 23, 28, 31, 45: no salen de ningún resultado.

Ahora no acepta una respuesta sin consultar antes.

Sin herramientas, el modelo **escribe como un analista**… de memoria. Por eso las fuentes las pone **el código**.

Todo junto: cifras de los datos y fuentes de los documentos

LO QUE PIDIÓ EL MODELO

consultar_datos

```
SELECT tienda, periodo, SUM(ventas_antes_reembolsos)
as ventas, SUM(abonos_reconocidos) as abon…
```

buscar_documentos

```
¿Qué cambios se implementaron en Malasaña a partir
del 30 de marzo?
```

Ojo: no consultó los pedidos de 4+ unidades (108 → 7), la cifra más reveladora.

Cifras clave: tras el 30 de marzo (periodo B), las ventas antes de reembolsos en Malasaña cayeron de 42.000 a 28.000 € [S1], las unidades vendidas de 467 a 304, y las ventas netas de 40.000 a 26.000 €, mientras los abonos se mantuvieron estables en 2.000 € [S1].

Hechos documentados: el 30 de marzo se retiró el cartel de la campaña Pack 4, que terminó oficialmente el 29, y se informó a los clientes de que el regalo ya no estaba disponible [D1][D2]. Hubo consultas sobre el regalo el 7 de abril [D1][D4].

Hipótesis: la retirada del cartel y la discontinuidad del regalo podrían explicar la caída en ventas, especialmente si el producto clave (Street One) tuvo falta de stock en tallas clave (41 y 42) en abril [D1][D4].

Falta: no hay datos de tráfico, consultas sin compra ni demanda no atendida, por lo que no se puede confirmar si la caída fue por pérdida de interés, falta de stock o ambos.

Datos para el cuánto (bloque I), **voz** y **documentos** para el porqué (bloques II y III).

Qué hace bueno a un RAG

✓ **Fuentes permitidas**
Una lista explícita. Nunca la carpeta del profesor.

✓ **Una ficha por fragmento**
Documento, página, fecha y estado (vigente o sustituido).

✓ **Citas y fuentes por código**
El modelo escribe [F#]; el código construye la lista.

✓ **Todo en local**
Embeddings, índice y modelo: los datos no salen.

✓ **Trocear por estructura**
Sección, página o turno; nunca a mitad de frase.

✓ **Búsqueda híbrida**
Significado y palabras clave; k pequeño (3–5).

✓ **Verificar y examinar**
Cifras contra fuentes; un examen con la fuente esperada.

PARA LLEVAR · BLOQUE III

Tres ideas para recordar

1 **Un RAG busca primero y responde después, citando.**
Solo con lo encontrado; cada frase, con su fuente.

2 **La similitud no sabe qué documento manda.**
Los metadatos (estado, versión, fecha) y el código, sí.

3 **Una cita real no garantiza un significado real.**
El código verifica la forma; una persona lee la fuente.

Datos, voz y documentos, en local y con fuentes: el analista de DropLab, completo.